

Java Training

Sử dụng class library

Package cơ bản

■ Library class

Là tập hợp các class bộ phận để tạo program

■ java.lang package

Class name	Giải thích
Object	Super class của tất cả các class. Định nghĩa chức năng chung mà object có.
String	Class để thao tác chuỗi ký tự.
System	Class xử lý input output tiêu chuẩn.v.v...
Math	Class xử lý hàm số toán học
Integer	Class để thao tác data kiểu int như là một object.

Object class

■ Object class

Là super class của tất cả các class. Định nghĩa chức năng chung của object.

● Ví dụ

```
Object obj1 = new Object();
System.out.println(obj1.toString());
Object obj2 = "Hello"; // Convert không tưởng mình subclass(String)→super class
(Object)

if (obj1.equals(obj2)) {
    System.out.println("obj1 và obj2 là object giống nhau");
} else {
    System.out.println(" obj1 và obj2 là object khác nhau");
}
```

Math class

■ Math class

Cung cấp field và method static để xử lý thao tác giá trị số như hàm lượng giác, căn bậc 2.

● Ví dụ

```
System.out.println(« lũy thừa 3 của 2 : " + Math.pow(2, 3));  
System.out.println(« khi làm tròn 1.56 thì:" + Math.round(1.56));
```

```
double d1 = Math.random();  
double d2 = Math.random();  
System.out.println("d1:" + d1);  
System.out.println("d2:" + d2);  
System.out.println(« lớn là" + Math.max(d1, d2));
```

String class

■ String class

Là class để thao tác chuỗi ký tự

Khi gán một chuỗi ký tự được bao bởi dấu nháy kép (") thì có thể sinh/tạo object.

● Ví dụ

```
String str1 = "abc";  
String str2 = "ABC";  
System.out.println(số kí tự của "¥"abc¥" : " + str1.length());  
System.out.println("charAt(1):" + str1.charAt(1));  
System.out.println("indexOf(¥'c¥'):" + str1.indexOf('c'));  
System.out.println("substring(1, 2):" + str1.substring(1, 2));  
System.out.println(« so sánh abc và ABC bằng equals:" + str1.equals(str2));  
System.out.println  
    (« so sánh abc và ABC bằng equalsIgnoreCase:" + str1.equalsIgnoreCase(str2));
```

StringBuilder class

■ Library class

Là class để xử lý chuỗi ký tự có thể thay đổi mà có method để edit string.

● Ví dụ

```
StringBuilder sb1 = new StringBuilder();  
System.out.println("sb1.capacity() : " + sb1.capacity());  
System.out.println("sb1.length() : " + sb1.length());
```

```
StringBuilder sb2 = new StringBuilder("abc");  
System.out.println("sb2.capacity() : " + sb2.capacity());  
System.out.println("sb2.length() : " + sb2.length());  
sb2.append("def");  
sb2.insert(3, "ABC");  
System.out.println("sb2:" + sb2.toString());
```

Wrapper class (1)

■ Wrapper class

Là class tương ứng với các kiểu data cơ bản.

■ Kiểu data cơ bản và wrapper class

Kiểu data cơ bản	Wrapper class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

Wrapper class (2)

■ Chức năng auto boxing

Là chức năng thực hiện convert tự động「kiểu data cơ bản → wrapper class」

● Ví dụ

```
Integer intObj = 5;
```

5 của kiểu int → Integer object thể hiện 5

■ Chức năng unboxing

Là chức năng thực hiện convert tự động「wrapper class → kiểu data cơ bản」

● Ví dụ

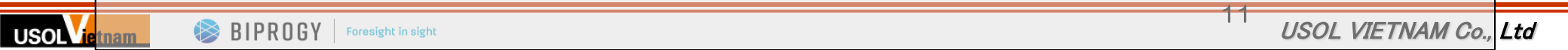
```
int num = intObj + 1;
```

Gán kết quả đã cộng thêm 1 vào giá trị int mà intObj quản lý vào num
Integer object → giá trị của kiểu int

Xử lý exception

■ Exception là gì

■ Các loại exception



Xử lý exception(1)

■ Xử lý exception là gì

Là cách xử lý đối với error phát sinh khi thực hiện/chạy

■ Các loại error và xử lý exception

Các loại error	Xử lý
Sub-class của Error	Việc xử lý exception là tùy ý (Thông thường, trong program có nhiều khi không thể xử lý được)
Sub-class của RuntimeException	Việc xử lý exception là tùy ý (Nếu xem xét lại program thì cũng có phần có thể xử lý được)
Là sub-class của Exception và không phải là sub-class của RuntimeException	Việc xử lý exception là bắt buộc (Nếu không mô tả xử lý exception thì có nhiều cái sẽ trở thành compile error)

Xử lý exception (2)

■ Xử lý exception

● Cú pháp cơ bản

「try~catch」 có thể nest (lồng)

```
try {
```

// Xử lý có khả năng phát sinh exception (①)

```
} catch (Kiểu của exception object   tên biến) {
```

// Xử lý khi phát sinh error (②)

```
} finally {
```

// Xử lý nhất định phải tiến hành dù có error hay không (③)

```
}
```

「catch(kiểu của exception tên biến)
{ }」 có thể chỉ định nhiều

「finally { }」 chỉ chỉ định trong trường
hợp cần thiết

● Trình tự xử lý

không phát sinh exception ① → ③ (② không được xử lý)

phát sinh exception ① (cho đến trước khi phát sinh exception) → ② → ③

※ Ở ② ghi trên, (dù có nhiều catch block), chỉ có 1 catch block tương ứng với class của exception object liên quan được xử lý.

Xử lý exception(3)

- Ví dụ phát sinh exception (sử dụng giá trị đã chỉ định khi thực thi để tính toán bằng 4 phép tính cộng trừ nhân chia)

```
>java Exception01 5 3
```

```
+:8
```

```
 -:2
```

```
 *:15
```

```
/:1
```

Trường hợp đã chỉ định 2 giá trị int có thể tính toán, thì hiển thị kết quả tính toán đúng

Chỉ định phần tử không tồn tại ở thao tác mảng

```
>java Exception01
```

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException : 0
```

```
at Exception01.main(Exception01.java:3)
```

Chỉ định giá trị không thể convert thành giá trị int

```
>java Exception01 a b
```

```
Exception in thread "main" java.lang.NumberFormatException : For input string: "a"
```

```
at java.lang.NumberFormatException.toString(NumberFormatException.java:48)
```

```
at java.lang.Integer.parseInt(Integer.java:447)
```

```
at java.lang.Integer.parseInt(Integer.java:497)
```

```
at Exception01.main(Exception01.java:3)
```

Xử lý exception (4)

■ Ví dụ về program

```
try {  
    int val1 = Integer.parseInt(args[0]);  
    int val2 = Integer.parseInt(args[1]);  
  
    System.out.println("+: " + (val1 + val2));  
    System.out.println("-: " + (val1 - val2));  
    System.out.println("*: " + (val1 * val2));  
    System.out.println("/: " + (val1 / val2));  
  
} catch (ArrayIndexOutOfBoundsException ex) {  
    System.out.println("cách sử dụng : java Exception02 số nguyên 1 số nguyên 2");  
  
} catch (NumberFormatException ex) {  
    System.out.println("Ở đời số hãy chỉ định giá trị số nguyên");  
    System.out.println("Cách sử dụng : java Exception02 số nguyên 1 số nguyên 2");  
}
```

Trường hợp phát sinh exception, thì thực hiện try { giữa chừng } → catch tương ứng

Trường hợp không phát sinh exception thì chỉ thực hiện xử lý try { }

Phát sinh exception !

Xử lý exception (5)

- Trường hợp muốn tập hợp các exception không cần xử lý riêng biệt để catch

Ở catch cuối cùng, chỉ định Exception class (super class của tất cả exception)

- Get thông tin error

```
try {  
    (lược)  
} catch (NumberFormatException ex) {  
    System.out.println("Ở đối số hãy chỉ định giá trị số nguyên");  
    System.out.println("Cách sử dụng : java Exception02 số nguyên1 số nguyên 2");  
  
} catch (Exception ex) {  
    // System.out.println("Thông tin error : " + ex.getMessage());  
    ex.printStackTrace();  
}
```

Xử lý exception (6)

■ Mệnh đề finally

Khi đã định nghĩa mệnh đề finally, thì nhất định được thực hiện mà không cần biết có error hay không

● Ví dụ

Trường hợp không phát sinh exception, thì chỉ thực hiện try { } và finally{ }

```
try {  
    (lược)  
} catch (NumberFormatException ex) {  
    (lược)  
} catch (Exception ex) {  
    (lược)
```

```
} finally {  
    System.out.println("finally nhất định phải được thực thi")  
}
```

Trường hợp phát sinh exception thì thực hiện try { giữa chừng }
→ catch(){ } tương ứng
→ finally{ }